Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

RUBY ON RAILS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on MVC, Active Record, routes, jobs, and testing

TOTAL: 10 QUESTIONS

Q1 What does 'Convention over Configuration' mean in Rails? Rails Model

Q2 What is the Rails request lifecycle? Architecture

Q3 What is Active Record and how do associations work? Configuration

Q4 How do Rails migrations work and how do you roll one back? Hacking

Q5 How does Rails routing work with resources? SSO / IdP

Q6 What are Rails concerns and what problems do they solve? Observability

Q7 How does Active Job work with Sidekiq? Lifecycle

Q8 What is Action Cable in Rails? Compliance

Q9 How does Rails asset management work with Propshaft? Testing

Q10 How do you test a Rails model with RSpec and FactoryBot? Training

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 **Rails derives behavior from naming conventions. A** Mitigation

Rails derives behavior from naming conventions. A User model maps to the users table. UserController handles /users routes. This means less config: no mapping files are needed. Follow the convention and Rails wires everything automatically; deviate and you configure explicitly.

A2 **A request hits the Router (config/routes.rb) which** Primitives

A request hits the Router (config/routes.rb) which dispatches to a controller action. The action interacts with models, then renders a view or returns JSON. Rack middleware (CSRF, session, logger) wraps the entire stack and runs before/after the router.

A3 **Active Record implements the active record pattern:** Hardening

Active Record implements the active record pattern: model instances map to rows and class methods generate SQL. has_many :orders in User and belongs_to :user in Order define the association. user.orders loads associated rows. user.orders.create(total:100) inserts and associates.

A4 **rails generate migration AddAgeToUsers age:integer creates a** Caching

rails generate migration AddAgeToUsers age:integer creates a timestamped migration file. rails db:migrate runs all pending Up methods. rails db:rollback runs the most recent Down method, reversing the change. db:migrate VERSION=20230101 migrates to a specific version.

A5 **resources :users in routes.rb generates seven RESTful** Federation

resources :users in routes.rb generates seven RESTful routes (index, show, new, create, edit, update, destroy). namespace :api {resources :users} prefixes routes and expects Api::UsersController. match '/about', via: :get defines a custom route.

A6 **A concern is a module in app/models/concerns** Observability

A concern is a module in app/models/concerns or app/controllers/concerns using ActiveSupport::Concern. Include them in models/controllers to share reusable behavior (Taggable, Auditable) without multiple inheritance. included do ... end runs in the including class's context.

A7 **Define a job class extending ApplicationJob with** Automation

Define a job class extending ApplicationJob with a perform(*args) method. Set queue_adapter :sidekiq in ApplicationJob. Enqueue with MyJob.perform_later(user_id). Sidekiq workers pick up jobs from Redis, call perform, and handle retries on failure with exponential backoff.

A8 **Action Cable integrates WebSockets into Rails using** Governance

Action Cable integrates WebSockets into Rails using channels. A channel class (ApplicationCable::Channel) defines subscribed and receive methods. The client subscribes via App.cable.subscriptions.create('ChatChannel', callbacks). Broadcast from server with ActionCable.server.broadcast.

A9 **Propshaft (Rails 7+) is a simpler asset** Assessment

Propshaft (Rails 7+) is a simpler asset pipeline. It fingerprints files for cache busting and serves them from /assets. Unlike Sprockets, it does not transpile or concatenate use importmap-rails for ES modules or a bundler (esbuild, Vite) separately for complex JS.

A10 **FactoryBot.define { factory :user { name 'Alice';** Optimization

FactoryBot.define { factory :user { name 'Alice'; email 'alice@ex.com' } }. In the spec: let(:user) { create(:user) }. Test associations, validations, and methods with expect(user.email).to eq('alice@ex.com') and expect(user).to be_valid. Run with bundle exec rspec.